A Survey of Methods in Song Detection and Music Information Retrieval

A thesis submitted in partial fulfillment
of the requirements for the degree of
Master of Science

Department of Mathematical Sciences
College of Arts and Sciences
University of Cincinnati, March 2026

Author: Nicky Zarick
Degrees: B.S. Mathematics, 2017,
Ohio State University

Chair: Deniz Bilman, Ph.D.
Committee: Yizao Wang, Ph.D.

Abstract

Identifying a song from a short, noisy recording without any metadata is a central problem in Music Information Retrieval, made difficult by environmental noise, uncertainty regarding the query's start time, and the large scale of music catalogs. This thesis studies a practical approach to song detection and extends it towards the harder task of cover song identification, where significant aspects of the song may differ from the reference recording.

We first present a robust baseline algorithm for exact recording identification, modelled after the *Shazam* system. Audio is transformed into a time-frequency representation using the Short-Time Fourier Transform, reduced to a sparse "constellation map" of local maxima, and encoded as individual hash-based fingerprints formed from small-scale maxima relationships. Matching is performed by retrieving hash collisions from a database and enforcing a temporal consistency constraint, reducing the impact of spurious matches and enabling song identification at a commercial scale.

We discuss the Non-Negative Matrix Factorization as a complementary tool for obtaining parts-based decompositions of audio signals, separating the spectrogram of an audio signal into interpretable spectral templates and temporal activations.

Finally, we present an adapted framework which is capable of handling cover detection by targeting invariant qualities persistent between versions of a song - namely, the dominant melody. The proposed pipeline isolates vocals using a source separation tool called Demucs as a means of identifying the dominant melody, converts the vocal track into chroma features to reduce sensitivity to timbre, introduces time quantization to handle timing differences, and expands the hash definition to recover discriminative power. We then present a simple implementation of the proposed algorithm, and end the discussion with a survey of other methods in cover detection.

© 2026 by Nicky Zarick. All rights reserved.

Acknowledgments

I would like to express my utmost gratitude to my advisor and thesis committee chair, Dr. Deniz Bilman. His continuous guidance and expertise proved invaluable throughout my time as a graduate student at UC.

I would also like to thank the Graduate Program Director and thesis committee member, Dr. Yizao Wang, for his support and advice during my education.

Thanks also to all the excellent professors of the department, who taught each class with a clear passion and care for their students. Similarly, thanks to the support staff of the Department of Mathematical Sciences, and to my fellow graduate students who welcomed me into their community.

Finally, I thank my family and friends for encouraging me to pursue my education, especially Liv Rickerd, without whom none of it would have been possible.

Contents

Abstract ii

Copyright iii

Acknowledgments iv

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 History of Song Recognition . . . 2
- 1.3 Organization of the thesis . . . 3

2 Audio Processing and Song Detection 5
- 2.1 Overview: The Problem of Song Detection . . . 5
- 2.2 The Solution of Shazam . . . 6
- 2.3 Audio Signal Processing . . . 7
  - 2.3.1 Digital Audio Signals . . . 7
  - 2.3.2 Discrete Fourier Transform . . . 8
  - 2.3.3 The Fast Fourier Transform . . . 9
  - 2.3.4 The Short-Time Fourier Transform . . . 11
- 2.4 Audio Fingerprinting . . . 12
  - 2.4.1 Problem Definition and Constraints . . . 12
  - 2.4.2 The Constellation Map . . . 13
  - 2.4.3 The Audio Fingerprint . . . 14
- 2.5 The Shazam Database . . . 19
  - 2.5.1 Populating the Database . . . 19
  - 2.5.2 Finding a match . . . 20

3 Non-Negative Matrix Factorization 24
- 3.1 Introduction . . . 24
- 3.2 Motivation . . . 25
- 3.3 Calculating the NMF . . . 29

3.4 Data Compression 33
3.5 Other Uses of the NMF 34

4 Source Separation **36**
4.1 Introduction 36
4.2 Demucs 37

5 Cover Detection **40**
5.1 Introduction 40
5.2 Shortcomings of the Existing Implementation 41
5.3 Algorithm Alterations 43
5.3.1 Demucs source separation 43
5.3.2 Chromagram 43
5.3.3 Time Quantization 44
5.3.4 Hash Expansion 45
5.3.5 Other alterations 47
5.4 The Algorithm Implemented 49
5.4.1 A toy example 49
5.4.2 Shortcomings 51
5.5 Other routes of research 52

Bibliography **56**

Chapter 1

Introduction

1.1 Motivation

The prospect of song detection is often taken for granted - when one is in a public place and he or she hears an unfamiliar song that they would like to hear again in the future, they may not think twice about using their smartphone to identify the tune. In many cases, the task can be completed within the span of a single minute. One such method is to use an application called *Shazam*, which is freely available on most modern smartphones. The app takes a short recording and often identifies the tune accurately, even in the presence of environmental noise. How does this kind of application work?

This thesis seeks to analyze the mathematical and computer science tools used to construct such an application. Then, we will discuss the possibility of extending its features to work not just for specific recording identification, but for alternative version (often called a "cover song") identification.

Shazam has been operating a music recognition service since 2002, and there are many others like it. We will explore each aspect of the music recognition process from beginning to end—including discussions of the mathematical framework underpinning song

recognition, as well as the practical considerations that are key to effective implementation of these tools.

Broadly speaking, song recognition is a task which seeks to match audio, in one form or another, with its metadata (often, the title or artist of a given song). In most cases worth consideration, one has only the audio—or, more often still, a small segment of audio—to use for identification purposes. The seminal question, then, is how a system can process or analyze the information in that small segment of audio so as to ascertain which song it belongs to. While there are many methods to solve this problem, this thesis will focus primarily on the "audio fingerprint" method which has become popular in modern times.

Before we discuss the details of audio fingerprinting, we will first acquaint the reader with the history of the field so as to understand how the current technologies that exist came to be. We will take a largely academic viewpoint regarding the development of this industry as we observe the scholarly papers which serve as its cornerstones, but it is important to consider that commercial factors undoubtedly played a role in the development of the field. While we will not discuss at length the commercial considerations of the field, we will let the success of *Shazam* serve as a reminder of the importance of the field.

1.2 History of Song Recognition

Widely considered the first systematic treatment of content-based audio retrieval, Wold et al. (1996) introduced the problem of audio classification based on an audio signal's *content*, rather than on its *metadata* as had been the norm at the time. At the time this paper was being written (the mid- to late-1990s), music databases were largely organized by textual metadata which was recorded by human data-labelers. Wold et al. asked the important question: What if one could categorize music by the perceptual features of the comprising audio signal? Such a categorization wouldn't require human data-labeling

and would serve as a way to access audio files by means other than text (i.e., accessing an audio file in the database by supplying a small clip of audio). Wold et al. (1996) did not fully solve the problem of noise-resistant song identification—in fact, it did not seek to—but it did lay the theoretical groundwork for the advances to come in the following decade.

Many modern approaches to song recognition can be traced back to the seminal work by Wang (2003), which detailed a powerful "audio fingerprinting" technique to be discussed at length in later sections. In this paper, Wang discussed not only the applied mathematical methods that form the theoretical backbone of the technology, but the practical considerations for deploying such a system at a large scale. The ideas presented therein formed the basis for *Shazam*'s technological success; however, the conceptual foundations of Wang's results can be seen in the work of Haitsma and Kalker (2002), which discussed a similar system of audio fingerprinting, albeit without the large-scale market perspective.

Advances in the field continue. While classical audio fingerprinting continues to be the dominant force in song recognition, more recent work such as Grondin, Jolicoeur-Martineau, Noël, et al. (2020) discussed the integration of neural networks and machine learning into existing frameworks. While machine learning is certainly a promising tool for song recognition, significant attention should be given to those methods which make do without it, especially considering how well they have performed, even before the advent of practical neural networks.

1.3 Organization of the thesis

Chapter 2 discusses audio processing and song detection. This will form the basis of our discussion as we come to understand the many challenges of building a practical song detection application. We will discuss the basic structure of the song recognition

system used by *Shazam*, by analyzing basic audio signal processing, *Shazam's* audio fingerprinting process, and the structure of the back-end database.

In Chapter 3, we discuss the Non-Negative Matrix Factorization, an important mathematical tool for obtaining parts-based decompositions of audio signals. The Non-Negative Matrix Factorization is a widely-used tool for representing all kinds of data, including audio data, in an interpretable manner.

Chapter 4 discusses source separation, the process by which instrumental and vocal tracks within a single song are separated into stand-alone audio files. We discuss a particular tool for source separation called Demucs.

Chapter 5 will discuss an extension of the *Shazam* algorithm which is capable of detecting alternative versions of songs. It builds on the *Shazam* algorithm, extending its capabilities. This discussion is followed by a small-scale implementation, and a brief overview of other methods in cover detection.

Chapter 2

Audio Processing and Song Detection

2.1 Overview: The Problem of Song Detection

In this section, we will discuss the "problem" of song detection—namely, the challenges faced by any system that seeks to automatically detect the identity of a song from a given audio clip.

Suppose you are in a cafe, and playing over the loudspeaker is a song which catches your attention: a song you have never heard before, but would very much like to know the title of, so that you can hear it again in the future. It would be very convenient if your smartphone could tell you what the song was called, and that is precisely the function of song recognition software. A smartphone can collect a quick audio snippet by performing a quick recording, and within the span of a few seconds, can produce a "best guess" for the song. Once one has experienced the ease by which this can be accomplished, it is tempting to take the technology for granted.

Let us consider the potential issues that may arise in this process, acting as barriers to effective song recognition. There are three principal issues to discuss.

1. A noisy signal: In the situation described above, there may be a large number of sources of extraneous noise added to the pure signal of the song being played over the loudspeaker—reverb from poor acoustics in the cafe, ambient noise from other patrons of the cafe, and even the poor recording quality of a smartphone microphone. Each of these degrades the "true" signal of the song, distorting information.

2. A short signal: One cannot be expected to record the whole, or large parts of, a song. By the time you have heard the song, decided you like it, and pulled out your smartphone, the song may already be half over. You may only be able to record a short audio snippet.

3. A large database of songs: There are, by some estimates, at least a hundred million songs in popular circulation that must be considered. This number grows with time, meaning that this issue gets more significant with each passing year. The storage required to store such a large corpus of music surpasses hundred of petabytes, which is likely to pose significant issues when trying to identify a single song among hundreds of millions.

In addition to these problems, others exist outside the purview of this thesis, including data compression considerations (some early song recognition systems operated over phone lines!) and licensing issues (who owns the legal right to a song?). While these are vitally important considerations for anyone who seeks to make a practical implementation of a song recognition system, they are not within the scope of this thesis and are thus ignored.

2.2 The Solution of Shazam

As previously mentioned, *Shazam* is a music recognition application that has existed since 2002, though it has changed form many times as it adapted to an increasingly capable technological landscape. What began as a niche over-the-phone service has become an

on-demand application, dominant in the music recognition space since its acquisition by Apple Inc. in 2018.

To understand how *Shazam* addresses the problems mentioned in the previous section, we will first develop a basic understanding of audio signal processing, before discussing *Shazam's* database and algorithmic setup, and then finally we will observe a practical application of the technology. By the end, we will have a solid grasp of audio fingerprinting, and the methods by which it is employed to identify songs, despite the above-mentioned hurdles.

2.3 Audio Signal Processing

Digital audio processing is a deep and impactful field. Any attempt to give a complete survey of the field would likely come up short, so instead we present a quick introduction to the tools which are relevant to the problem of song identification. We will examine various forms of the Fourier Transform, which plays an integral part in many aspects of digital audio processing. In our case, the motivation behind employing this transform is singular: to produce a representation of an audio file known as a spectrogram. The spectrogram will form the basis of the acoustic fingerprinting technique which will be discussed in the next section, and as such it is of vital importance to understand how the spectrogram is produced in the first place.

2.3.1 Digital Audio Signals

Digital audio is any representation of sound in digital form. Sound, in the physical realm, is simply a continuous variation of air pressure. This can be modelled mathematically by a time-dependent function $f(t)$ whose output corresponds to these variations in air pressure. A primary consideration of digital audio processing is converting this continuous function of the real-world into a discrete form that a computer can store and operate on.

Recording an audio signal and storing it digitally therefore requires the discretization of the function $f(t)$ into a sequence of datapoints $f[n]$.

The **sampling rate** determines the resolution of this discretization, and serves to answer the question, "how many discrete chunks should the continuous signal $f(t)$ be split into?". Sampling rates are most often measured in samples per second (Hz), with common values being 44.1 or 48 kHz. A higher sampling rate allows the discretization $f[n]$ to capture more finely-detailed variations in the original signal $f(t)$.

This discretization forms the basis for understanding digital signals. However, if we wish to *do something* with the audio signal, to analyze or transform its contents in a meaningful way, we would be wise to use a powerful tool called the Fourier Transform.

2.3.2 Discrete Fourier Transform

The **Fourier Transform** is an integral transform taking a continuous function $f(t)$ as input, and outputting a continuous function of frequency $\hat{f}(\xi)$. We can define the Fourier transform in the following way:

$$\hat{f}(\xi) = \int_{-\infty}^{\infty} f(t)\, e^{-2\pi i \xi t}\, dt,$$

where ξ is a given frequency. The resulting function $\hat{f}(\xi)$ describes the extent to which various constituent frequencies are present in the original signal. It should be noted that the output of this transform is a complex-valued function, since we are capturing both the *magnitude* and the *phase* of the constituent frequencies.

This is a powerful and widely-used transformation, but we must remember that, though our original audio signal can be viewed as a continuous time-dependent function, our actual digital audio file is a discretized version of the original function. Thus, we cannot use the above formulation of the Fourier Transform if we wish to decompose our digital

signal into its constituent frequencies—a vitally important task for song identification, as we will see in later sections. Instead, we look to the Discrete Fourier Transform.

The **Discrete Fourier Transform** is a discrete transformation analogous to the one above. It is defined similarly:

$$\hat{f}[k] = \sum_{x=0}^{N-1} f[x]\, e^{-2\pi i k x/N}, \quad k = 0, 1, \ldots, N-1$$

where k indexes frequencies, and N is the (necessarily finite) number of discrete intervals into which the original signal $f(t)$ was segmented. By convention, a discretized function is denoted with square brackets instead of parenthesis.

It should be noted that, while the continuous Fourier Transform can theoretically capture infinitely-high frequency resolution, the discrete form of the transform is limited by its sampling rate. This means that the information one can gather from the discrete Fourier Transform is limited by the sampling rate.

Since the Discrete Fourier Transform is made out of practical consideration, let us also examine the computation complexity of calculating $\hat{f}[k]$. For each output frequency k, we must sum over all N samples. Since there are precisely N frequencies, it is clear that the complexity of this problem is $O(N^2)$. For modern sampling rates, we expect N to be at least 4.41×10^4, bringing the total number of computations to around 1.94×10^9. Performed regularly, this may prove to be an unacceptable computational complexity—luckily, there is an alternative formulation of the transformation which allows a significant reduction in complexity.

2.3.3 The Fast Fourier Transform

The **Fast Fourier Transform** (FFT) is an algorithm which enables the computation of a Discrete Fourier Transform with only $O(N \log(N))$ complexity. This is achieved

by segmenting the transformation into smaller parts and taking advantage of certain symmetries. Observe this alternative formulation of the discrete transform:

$$\hat{f}[k] = \sum_{x=0}^{\frac{N}{2}-1} f[2x]\, e^{-2\pi i k(2x)/N} + \sum_{x=0}^{\frac{N}{2}-1} f[2x+1]\, e^{-2\pi i k(2x+1)/N},$$

which is precisely equal to our previous definition for $\hat{f}[k]$, with the indices of summation separated into an even and an odd part.

Rearranging, we can write:

$$\hat{f}[k] = \sum_{x=0}^{\frac{N}{2}-1} f[2x]\, e^{-2\pi i k x/(N/2)} + e^{-2\pi i k/N} \sum_{x=0}^{\frac{N}{2}-1} f[2x+1]\, e^{-2\pi i k x/(N/2)}.$$

Thus, we can see that the two finite sums:

$$\sum_{x=0}^{\frac{N}{2}-1} f[2x]\, e^{-2\pi i k x/(N/2)}$$

and

$$\sum_{x=0}^{\frac{N}{2}-1} f[2x+1]\, e^{-2\pi i k x/(N/2)}$$

are simply index-shifted versions of the same computation, allowing us to save time and computation by calculating these values only once for each index.

This splitting and grouping can be applied recursively, halving the length of the summed index. Each recursion level carries out $O(N)$ multiplications, and there are $\log_2(N)$ total levels to go through. Thus, the FFT has a total complexity of $O(N \log(N))$

The algorithm described above has been known since well before its use in song identification. It has been known since 1965, and it is called the Cooley-Tukey FFT algorithm after the two scientists who first formulated it. While we only provide a surface-level glance into the algorithm itself, more can be seen in Cooley and Tukey (1965). In practice, we often use this algorithm without giving it much thought.

Thus, we have reduced the total complexity of this task from $O(N^2)$ to $O(N\log(N))$—a significant reduction!

There remains, however, one point to consider. When taking a Fourier Transform, we define some time-interval over which to perform the integral or sum, but it would be quite useful to see how the frequency content of a signal changes from moment to moment. Ideally, one would like to know the frequency components of a signal at each time t (or each sample n) but integrating or summing over a single point will not yield a meaningful answer. This leads us to an important conclusion to our discussion of Fourier Transforms: the Short-Time Fourier Transform.

2.3.4 The Short-Time Fourier Transform

The **Short-Time Fourier Transform** (STFT) addresses the above-mentioned problem by computing Discrete Fourier Tranforms over short, overlapping windows of time. This produces a "snapshot" of the frequency content for a given time bin, effectively producing a two-dimensional representation of the intensity of a spectrum of frequencies over time.

Observe the formulation of the STFT:

$$X(m,k) = \sum_{n=-\infty}^{\infty} f[n]\, w[n-m]\, e^{-2\pi ikn/N}$$

where $f[n]$ is the discrete signal, $w[n]$ is a window function which "selects" short windows of time, and m indexes the time frame.

The window function $w[n]$ is often chosen to be bell-shaped (such as the Hann window function) and effectively determines the outcome of an important tradeoff: shorter windows (with respect to time) offer a higher time-resolution but a lower frequency-resolution. The opposite is true for longer windows. This tradeoff is fundamentally unavoidable, reflecting the uncertainty principle inherent to frequency analysis.

Despite this trade-off, the STFT provides a valuable glance into the frequency makeup of an audio signal over time. By observing the magnitudes of those frequencies across

time, we obtain an important object called a **spectrogram**. This 2-dimensional plot shows us which frequencies have which magnitude for any given time. Spectrograms, and the data we derive from them, form the foundation upon which the entire method of audio fingerprinting is built.

2.4 Audio Fingerprinting

2.4.1 Problem Definition and Constraints

It may be tempting to view a spectrogram as a final, complete representation of a song. If one has the spectrogram of a song and a database of spectrograms, then a simple comparison between the spectrogram and the database would prove to be a simple way to identify a song. However, recall the the problems discussed in Section 2.1. Any noise present in the signal, or distortion of the original signal, will produce noticeable variations in the recorded signal, and therefore will change the spectrogram. Thus, a naive comparison will not do, if we hope to have a system resistant to noise and signal distortion. Moreover, a spectrogram is a 2-dimensional matrix representation of the STFT's magnitude at each time in a song. This will prove inefficient when comparing the recorded signal against a database with hundreds of millions of entries. Finally, we must consider that the recorded audio clip is a short snapshot of the full song. This means that the recorded signal must be compared against not only each song in the database until a match is found, but each subsection of each song, where the subsection length matches that of our recorded signal. In other words, we do not know from which part of a song the recording is taken, so we must account for this when comparing against the database.

Thus, the need for a representation of a song which is simultaneously resistant to noise, more computationally light-weight than the spectrogram, and time-localized is made

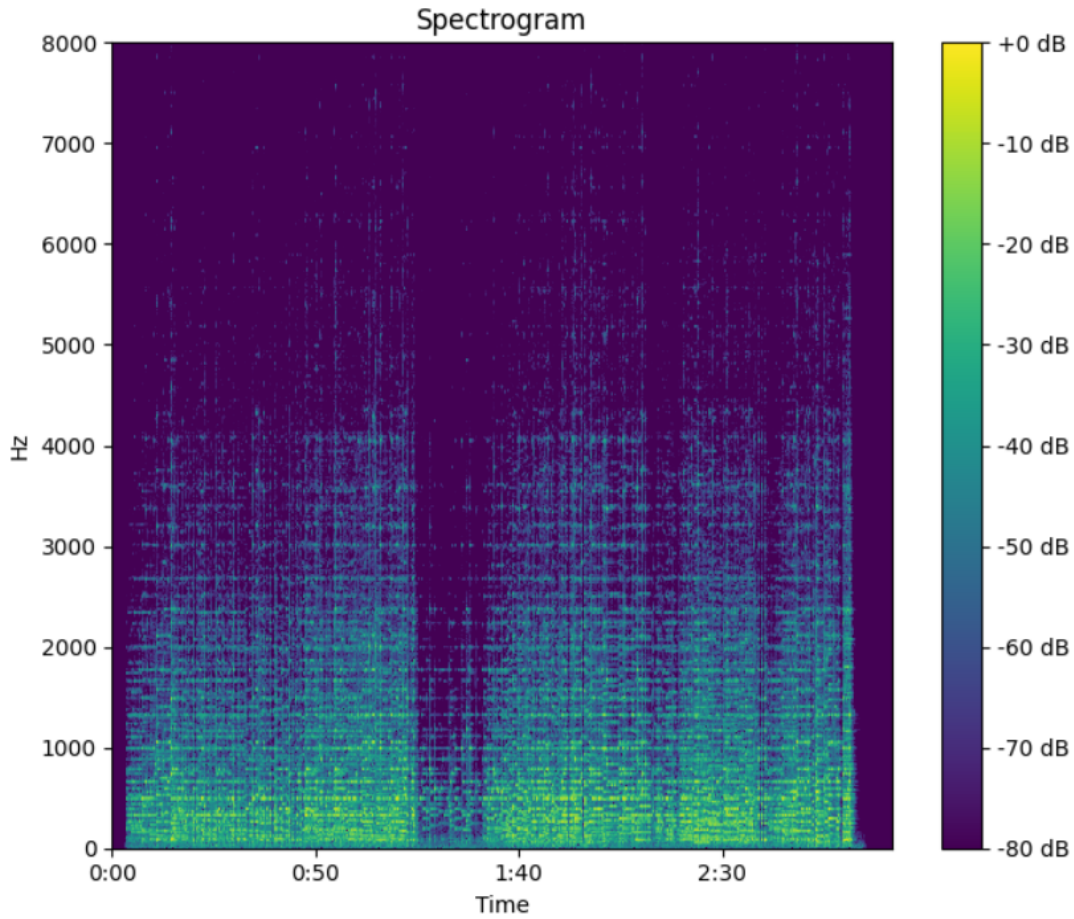

Figure 2.1: The spectrogram of a 3-minute song, showing frequency intensity over time. Note the change in frequency makeup over time, as well as the higher intensities in lower frequencies.

clear. With the constraints of our problem at hand, we can begin to construct the audio fingerprint, seeking to resolve these shortcomings.

2.4.2 The Constellation Map

The first step is to reduce a spectrogram from a dense matrix to a sparse set of coordinates. We do this by converting it into a **constellation map**, constructed by considering only the high "peaks" of the spectrogram. That is, a time-frequency point is a candidate "peak point" if it has a higher magnitude than all of the other time-frequency points within a small neighborhood. In this way, we can form a "constellation" of peaks which essentially

describe the most intense representations of varying frequencies at different times. The exact threshold for whether or not a time-frequency point is considered a peak should be adjusted so as to maintain a consistent density of peaks across spectrograms of varying content. In other words, the threshold a time-frequency point needs to meet in order to be considered a peak should be higher in signals which have, on-average, more intensity. The reverse is true for signals which have lower intensity.

Let us observe that, at this point, any data regarding amplitude is discarded. The constellation map does not record the amplitude of a peak, rather only that a given point is or is not a peak.

The primary motivation for selecting peaks using this kind of threshold is exactly this: the time-frequency points with the highest magnitude are the most likely points to be resistant to added noise and signal distortion. A secondary, but still important, motivation for selecting peaks in this way is to significantly reduce the size of the data required to represent the signal. The constellation map reduces a dense, large matrix (the spectrogram) into a comparatively shorter sequence of 2-dimensional coordinates. Furthermore, the length of this sequence can be adjusted as needed, simply by altering the adaptive threshold rule used to select the peaks.

2.4.3 The Audio Fingerprint

Constructing an audio fingerprint out of a constellation map is the final step in creating a representation of digital audio that is both lightweight and resilient to noise and distortion.

We cannot use the individual frequency-time pairs comprising the constellation map for database comparison. There are a few reasons for this: consider that many songs will have peaks in similar places, and that small amounts of noise or signal distortion may create "false peaks", or suppress the presence of "real peaks" within the constellation map. A direct comparison between a recorded signal's constellation map and a database

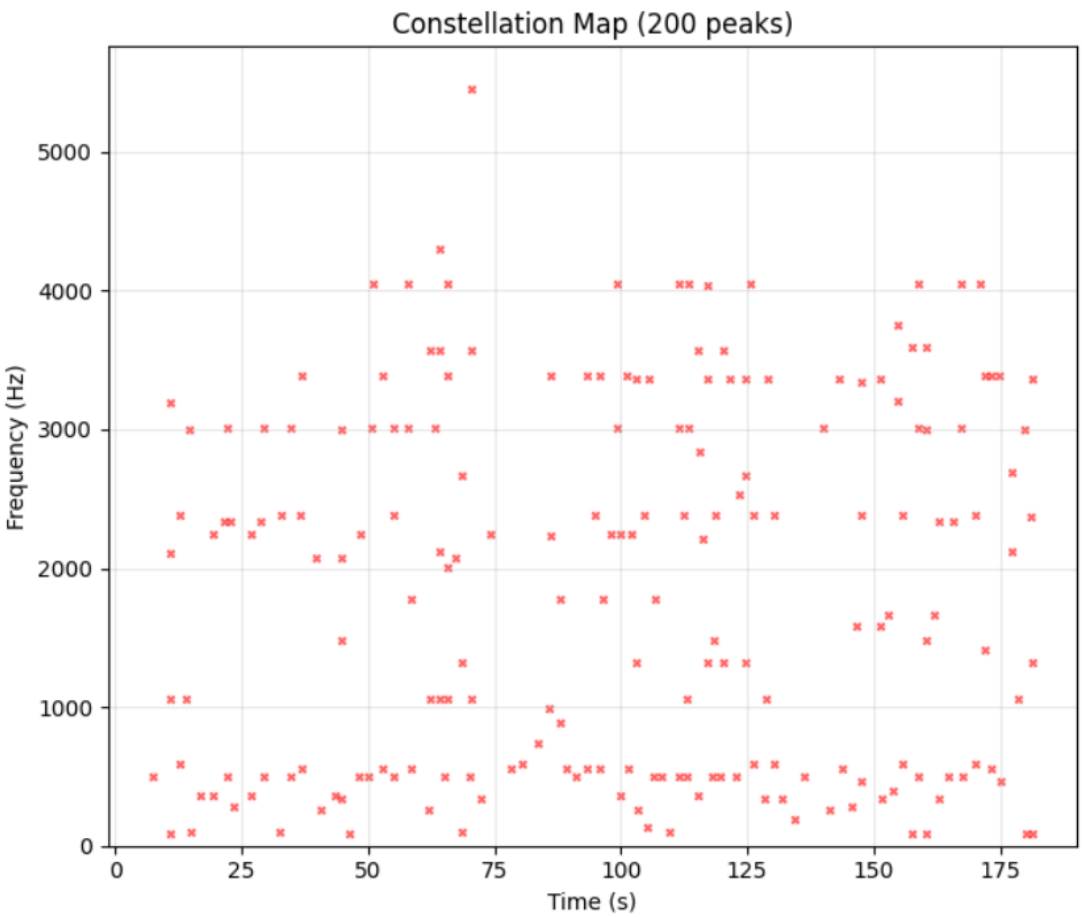

Figure 2.2: A constellation map, derived from the spectrogram shown in Figure 2.1. Note the difference in y-axis scale. As is common in spectrograms of songs, the highest frequencies tend to have a lower density of intensity peaks, if they have any at all.

would not have the resilience against noise that we seek, as even small differences caused by noise would result in a significant mismatch during pointwise comparison.

While constellation maps seek to reduce the amount of noise present in our representation of the signal by discarding much of the lower-intensity information, it is perfectly possible that there are "false peaks" present in the constellation map due to noise or distortion. As such, our ultimate solution must be robust even to the presence of these erroneous peaks. To accomplish this, we will consider the *distances between collections of nearby peaks* within a small region of the constellation map. The outcome is that we translate a problem of absolute geometry (where each individual peak is within a constellation map) to relative geometry (which peaks are close to one another, and how close they are).

We first select a set of points in the constellation map to act as anchor points—often using local maxima, which tend to have a lower likelihood of being the result of errant noise within the signal. For each anchor point, we look within a nearby shifted-time window for other peaks and calculate their distance (in the time domain) from the anchor point. We record the frequencies of these points and their relative distance in a triplet called a hash:

$$h_{i,j} = (f_i, f_j, \Delta t)$$

where $h_{i,j}$ is a hash, f_i is the frequency of the anchor point, f_j is the frequency of the other constellation point (called a "target point"), and Δt is the difference in time between them. We store the hash $h_{i,j}$ alongside the time value of the anchor point, t_n.

Generally, for a given anchor point f_i, f_j is chosen within some constraints relative to f_i. These constraints are made with respect to time and frequency: some positive-valued parameters Δt_{min} and Δt_{max} are chosen to restrict target points to an interval succeeding the anchor point. Moreover, some parameter Δf is chosen restrict target points from occurring too far (in the frequency domain) from the anchor point, and this restriction

is applied in the positive and negative direction. These two restrictions—time and frequency—form the bounds of a "window" nearby (but not overlapping with) the anchor point from which target points may be selected for fingerprinting.

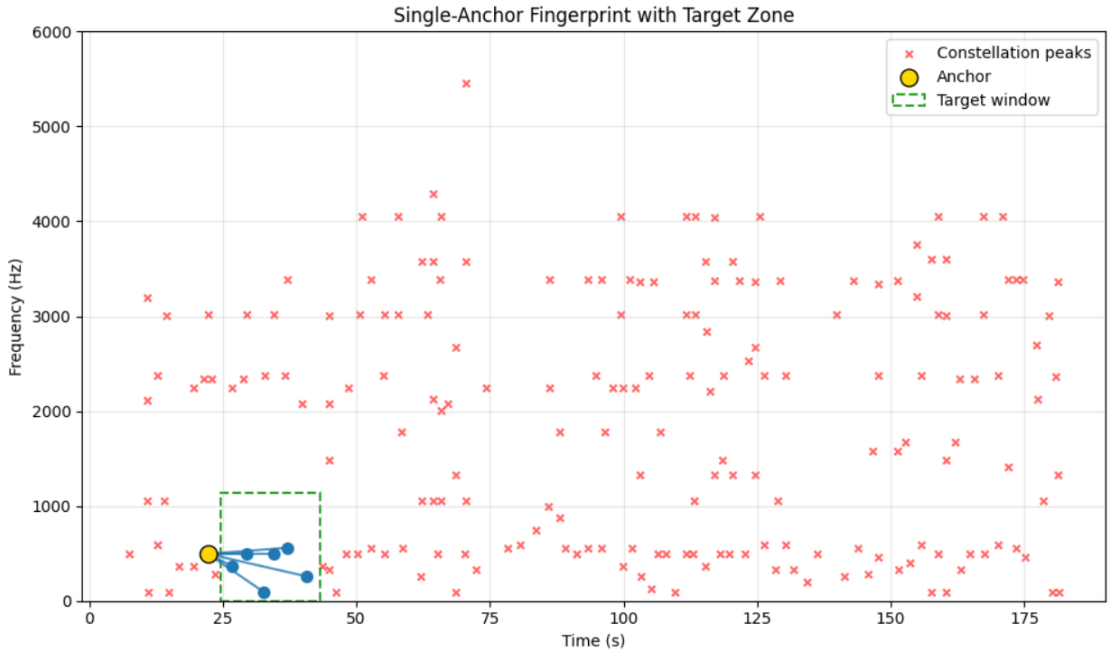

Figure 2.3: A single anchor point, connected to various target points within the target window defined by bounds on time and frequency, relative to the anchor point.

For a given anchor point, we construct many hashes, (depending on the parameters of our hashing algorithm) though we often define some limit for the number of hashes derived from a single anchor points so as to not over-rely on any given point in the signal. Furthermore, we have many anchor points. Our fingerprint is thus a collection of N triplets and the time at which each hash's anchor point occurred, t_n. In summary, the fingerprint, $F_{i,j}$, identified by two frequency bins i and j, will take the form:

$$F_{i,j} = [(f_i, f_j, \Delta t), t_n]$$

which can be written more compactly as:

$$F_{i,j} = [h_{i,j}, t_n]$$

In practice, an upper limit on $|f_i - f_j|$ is set so as to keep the length of a fingerprint minimal.

This fingerprint is valuable insofar as it encodes the relative geometry of peaks in the constellation map. This serves the purpose of noise-resilience well: even if noise removes peaks or adds spurious ones, it is unlikely to remove all anchor-target pairs.

A vital point is that, when matching this fingerprint against a database, one does not need to look for an exact match between the fingerprint's list of hashes and the database. In fact, such an exact match is quite unlikely to exist in the presence of any noticeable noise. Instead, one only needs to look for a "closest match", comparing the hashes in the recorded signal's audio fingerprint against the hashes in the database. This is precisely how the fingerprint remains robust to noise.

The fingerprint has another significant advantage over the constellation map: time-shift invariance. While constellation map points (t, f) use absolute time, hashes in the fingerprint use relative time Δt. Thus, hashes can be matched against a database regardless of where the recorded audio clip occurs within the original song (recall that we likely have no information regarding where in the song the audio clip comes from). Constellation maps, on the other hand, would require a more complex comparison: since the recorded audio clip can come from any part of the song, one would need to check it against varying time-offset sections of each song within the database for a match. This is computationally expensive and time-consuming, especially when scaled to database with hundreds of millions of entries.

The advantage introduced by fingerprinting thus becomes clear: the time-shift invariance of the fingerprint allows hashes to match against a database regardless of where they occur in a song, and time-alignment can be recovered by comparing anchor times.

Up to this point we have said little about the database itself. Let us consider, in the following section, the creation and structure of such a database, which will make the value of audio fingerprinting clearer.

2.5 The Shazam Database

2.5.1 Populating the Database

In order for a song to be identified, it must of course be in the database—but in what form? Given the process described in the previous section, it is obvious that the raw audio file will not be practical or sufficient. Instead, the process described above must be carried out on each of the songs that we wish to have in the database: perform a STFT on the digital audio file to produce a spectrogram, create a constellation map, and extract a collection of hashes in the form of an acoustic fingerprint. This will produce, in effect, a very long list of hashes, which should each be associated with a song title and the time within that song that the hash occurs.

Furthermore, there is a significant amount of metadata related to each song that should be recorded somewhere: the artist name, the album title, where to purchase or listen to the song, and perhaps even more metadata related to licensing which may be important when considering commercial applications of the technology. Ultimately, most of this extra metadata can be stored in a separate database from the one described here, with the song title (or, more practically, some unique ID for the song title) acting as the join key. For simplicity we will assume that the "extra" metadata is stored elsewhere, while we only consider the hashes, their song IDs, and their timestamps.

In summary, we construct a large database of triplets containing hashes (which, you may recall, are themselves triplets), song IDs, and timestamps. Once the database is compiled, one can make a short recording and attempt to find a match.

2.5.2 Finding a match

Observe a single hash obtained from a recorded sample, which we can model, as above, as a triplet: $(f_i, f_j, \Delta t)$. When checking this hash against a database, we may return many matches in the form of tuples comprised of a song ID and a timestamp where that hash occurred in the song. This may look like:

$$
\begin{gathered}
(f_i, f_j, \Delta t) \text{ returns:} \\
[(\text{song_47}; t = 24.3\text{s}) \\
(\text{song_93}; t = 1.9\text{s}) \\
(\text{song_03}; t = 43.5\text{s}) \\
...]
\end{gathered}
$$

Each hash from the recorded clip's fingerprint will produce some number of matches in the database, each described by a specific song and a timestamp. In other words, the specific hash $(f_i, f_j, \Delta t)$ is found in both the recorded signal and the database.

With such a large database, it is inevitable that a single hash will return many results, so that we expect many song IDs to be returned for any hash. A naive approach to identifying the song in question would be to simply count the number of times each song ID shows up across all hash database queries and select the one with the most occurences. While this may produce some degree of success, it would ignore the temporal structure that is captured in the fingerprint. Indeed, one should make use of the temporal consistency across multiple independent hash lookups to increase the confidence that a

true match has been found. Recall that the hash $(f_i, f_j, \Delta t)$ is recorded alongside its time occurence within the recorded audio clip, t_i. In a true match, the time difference between hashes should be the same time difference observed between database results.

For example, consider two hashes in a given audio clip's fingerprint: $F_{1,1}$ and $F_{2,1}$ with associated time stamps t_1 and t_2. Note that these two hashes are sourced from separate anchor points so that $t_1 \neq t_2$. Suppose also that both hashes return database hits for song_47. Then, if song_47 is indeed a true match for our recorded audio clip, one expects the time difference between the two database hits to be precisely (or very nearly) equal to $t_2 - t_1$. In this way, we impose some time-structure requirements on our potential database matches. Simply returning many hits for a given song is not enough to determine a true match—the hits must have the same temporal structure as those in the recorded clip.

Note that the presence of noise in the recorded clip may introduce erroneous database hits: The false peaks in the constellation map created by this noise create erroneous hashes, which in turn create erroneous database hits. This is no problem for the database in question; these false database hits will *not* align with the time structure of true hashes, and thus will not be considered when counting up database hits. In fact, when faced with a collection of potential database hits, one can graph database time and query time on separate axes. In such a graph, true hits can be expected to form a straight line with slope exactly equal to 1, since the plotted points are expected to have a consistent offset on both axes (see Figure 2.4).

Thus, we have accomplished our initial goal of creating a song-detection system which:

1. is resistant to noise and signal distortion,

2. is time-invariant,

3. capable of identifying one song out of a database of hundreds of millions.

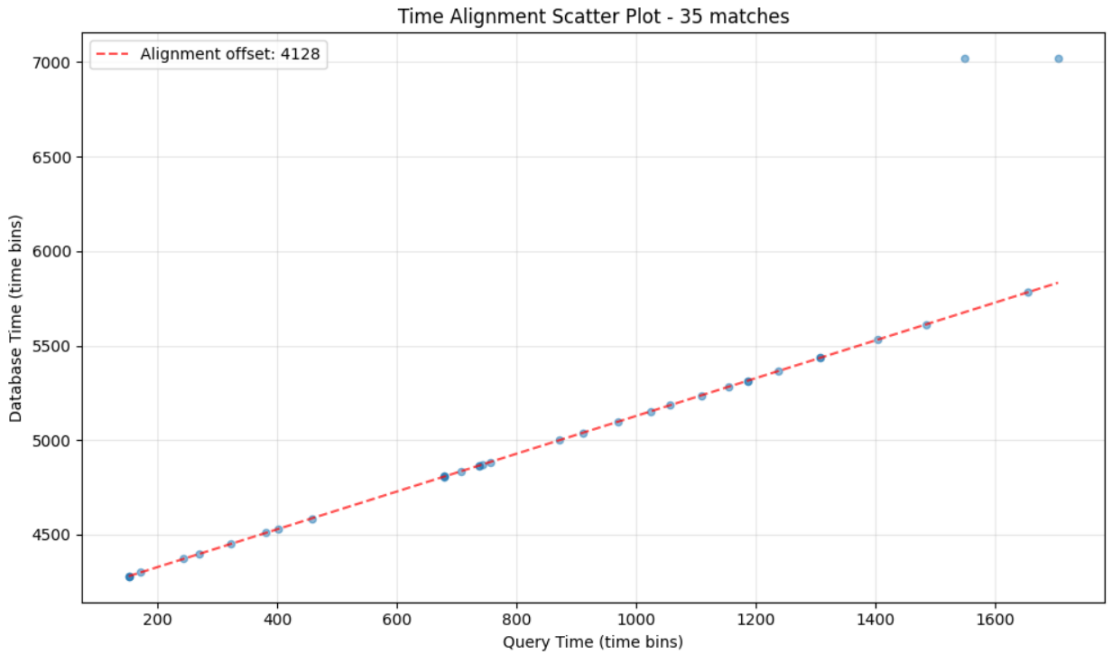

Figure 2.4: Database matches plotted on the time-axes of the query time and database time offsets. Each blue dot corresponds to a hash found in both the query and the database. Note that most of the dots are situated on a straight line. Two erroneous database hits (in the top-right corner) are not considered because they are not time-aligned consistently with the rest.

To the last point, there may be some work left to do—while the process described thus far has certainly reduced the complexity of the problem at hand, there are many additional considerations one ought to make when dealing with a database of this size. Given a hundred million songs, each with, say, $10,000$ fingerprints, the database will swell to over a trillion fingerprints. However, the fingerprinting approach is inherently parallelizable since each fingerprint's query to the database is independent of the next. This makes the system inherently amenable to distributed architectures like sharding, which are all but required at a commercial scale. Rather than storing all of the fingerprints on a single machine and searching through them one-by-one, methods like hash-based sharding can help tremendously to break up this massive problem into many smaller ones. In sharding, both the database and a user's query are split up between many machines. A lengthy treatment of sharding is outside the scope of this thesis, but for an analysis of

the technique, challenges faced when employing it, and a survey of related tools, see Solat (2023).

Chapter 3

Non-Negative Matrix Factorization

In this chapter we will give an overview of the Non-Negative Matrix Factorization, including its uses, interpretability, and computational methods.

3.1 Introduction

The **Non-Negative Matrix Factorization** (NMF) is rather straightforward. Simply take any $n \times m$ matrix V whose entries are all non-negative real numbers, and factor it into two matrices, W and H, which also have purely non-negative real entries: $V = WH$. In this section, we will observe the means by which the matrices W and H can be calculated given an arbitrary non-negative matrix V, and we will explore some interesting properties of the NMF as they apply to the other sections of this thesis.

The seminal paper regarding the Non-Negative Matrix Factorization is Lee and Seung (1999), which focused on "parts-based" representations of data objects. Using examples mainly related to facial recognition, the authors explored how the NMF can extract features like noses, eyes, and mouths from images of faces. This differs significantly from other, similar methods that do not impose a non-negativity constraint, such as Principal

Component Analysis or Vector Quantization, which tend to reveal global, rather than local, features.

In the next section, we will lay the mathematical foundation for applying the Non-Negative Matrix Factorization to representations of sound, and further explore this "parts-based" representation of data as it applies to digital audio signals.

3.2 Motivation

In previous chapters, we have discussed the various uses of spectrograms, which can be seen as a time-versus-frequency visualization of digital audio. Recall that spectrograms are constructed using the Short-Time Fourier Transform to produce a 2-dimensional rectangular data structure, which has some intensity value at each intersection between the time-axis and the frequency axis. In other words, time and frequency are discretized into "bins" and where a time bin and frequency bin intersect, there is some intensity describing the amount of that frequency at that time. Such a data structure is very naturally stored in a matrix, whose rows describe a given frequency over time, and whose columns describe the frequency makeup of a single time-step.

Also note that the *intensity* of a sound is inherently non-negative. In more simple terms: sound intensity can be loud (large intensity), quiet (small intensity), or nonexistent (zero intensity). For this reason, it is natural to represent an audio clip using a non-negative time–frequency representation such as a magnitude (or power) spectrogram, whose entries record the strength of each frequency bin at each time bin.

Sound intensity is also commonly displayed in *decibels* (dB), a logarithmic unit that measures intensity *relative* to a chosen reference level. Under such a convention, values may appear negative when the intensity falls below the reference (for example, when the reference is chosen to be the maximum intensity of the signal). While dB-scaled plots are useful for visualization, the Non-Negative Matrix Factorization is most naturally applied to spectrograms in linear units. Accordingly, throughout this thesis we will take V to be

a non-negative spectrogram matrix (e.g., a magnitude or power spectrogram), so that $V \geq 0$ entrywise.

At this point, it may not be immediately obvious *why* one would go through the trouble to ensure the spectrogram can be represented by a non-negative matrix. The answer becomes clear when one analyzes the structure of the matrices W and H, which often exhibit notable patterns. For now, let us observe a visualization of the three matrices in question: V, W, and H in Figures 2.1, 3.2, and 3.3 respectively.

For purposes which will become clear soon, we call the W matrix the "Note Matrix", and the H matrix the "Activation Matrix".

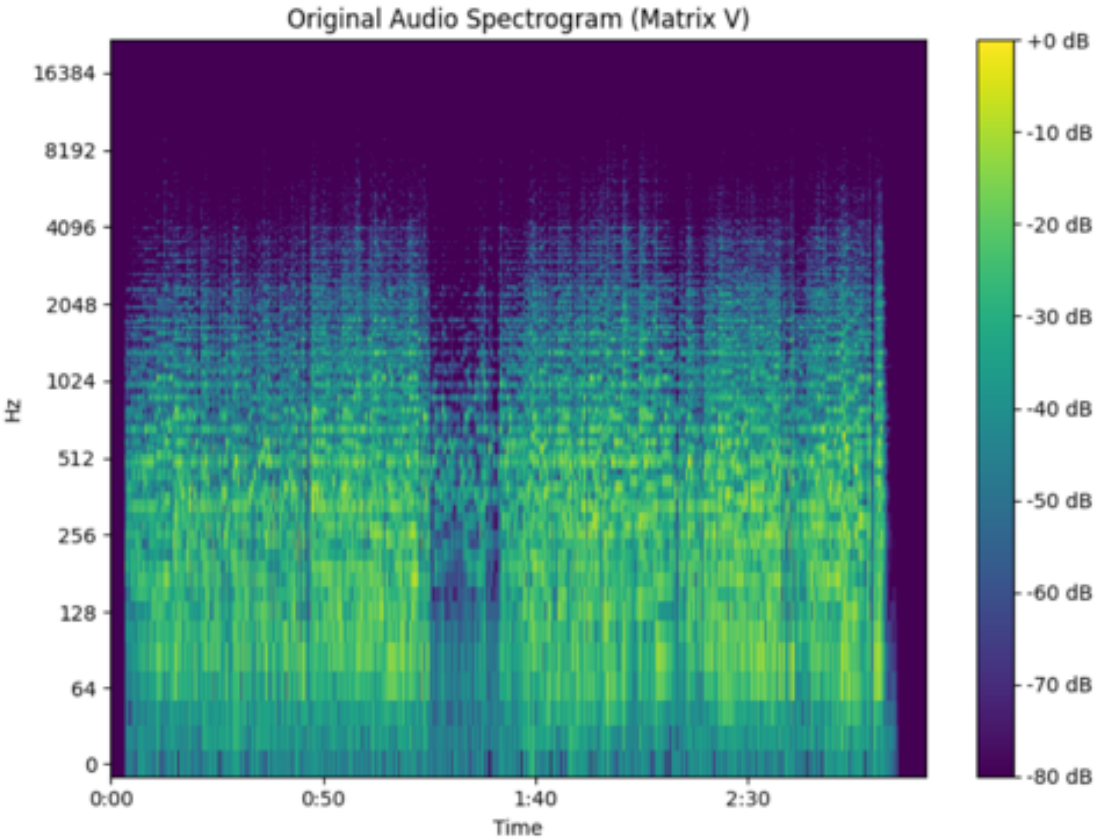

Figure 3.1: The spectrogram of a 3-minute song, the matrix V. Note the intensity values are negative. These are shifted up for the purposes of performing the NMF.

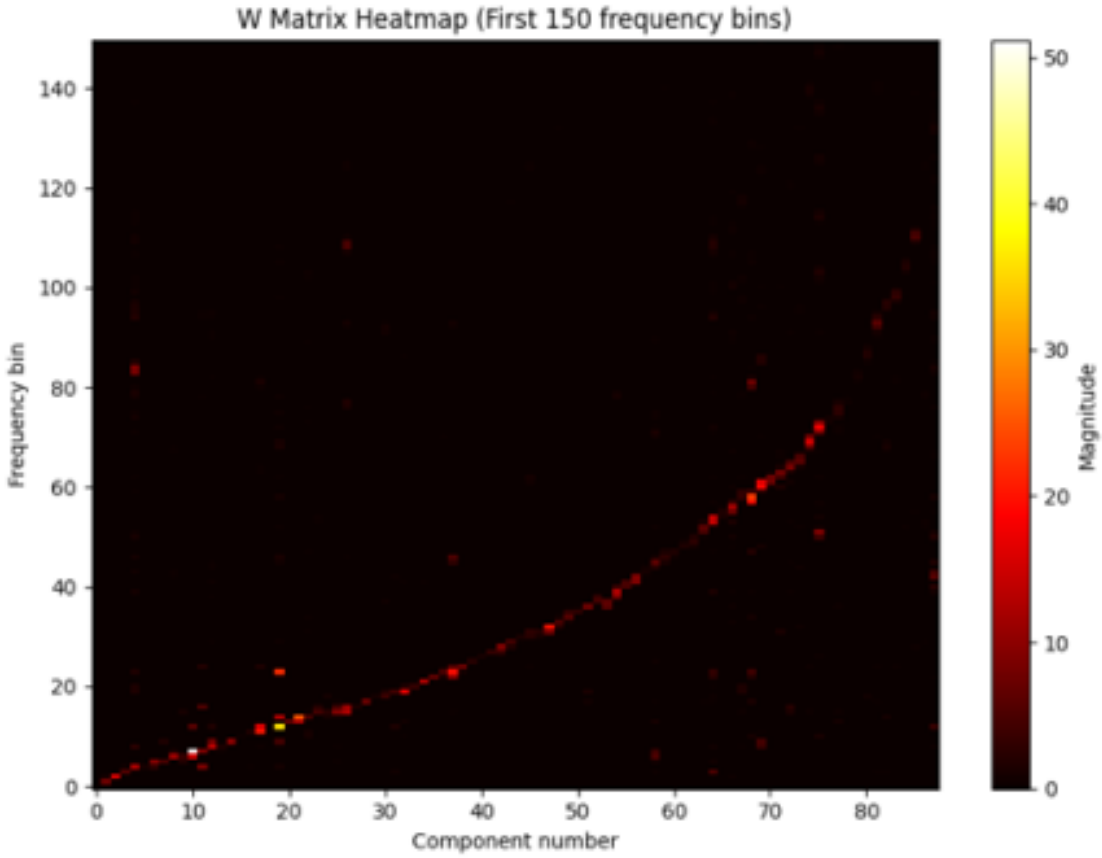

Figure 3.2: The W matrix visualized. Notice that most of the components are 0, while the non-zero components form some obvious structure. These individual components represent atomic elements of sound.

In short the W matrix (the Note Matrix) represents a spectral pattern of frequency while the H matrix (the Activation Matrix) represents *when* those frequencies occur. More specifically, we can interpret the Note Matrix as comprising the timbre, notes, and harmonic structures present in the spectrogram. Note, in Figure 4.2, that the y-axis is frequency and the x-axis is "component number". Thus, each non-zero element in the Note Matrix can be seen as a building block of the full song, which, when organized properly, produces the original full song. The schematic that defines that organization is precisely matrix multiplication by the matrix H, the Activation Matrix. In this sense, the Activation Matrix determines which components of the Note Matrix are "played"

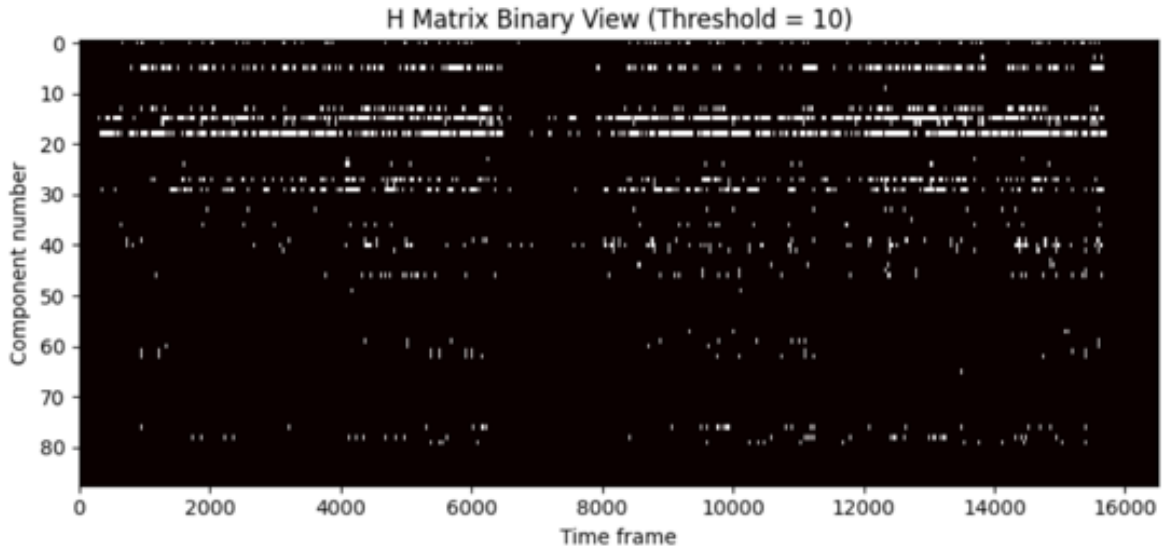

Figure 3.3: The H matrix visualized. A threshold function is applied here to make it more readable. As in the W matrix, there appears to be some obvious structure related to the underlying song. Each of the components from the H matrix are activated at specific times, hence the name "Activation Matrix".

and when. Thus we interpret the Activation Matrix (shown in Figure 4.3 in binary form) as a kind of "score" for the full song. Notice, in Figure 4.3, that the y-axis is "component number" while the x-axis is "Time". Thus, the Activation Matrix connects the components from the Note Matrix to various times.

Ultimately, we view the matrix factors of W and H as two critical pieces of information about the song in question: what notes are played (the Note Matrix) and when they are played (the Activation Matrix). The "notes", in this case, are not explicitly-defined as they might be in the traditional conception of a music score, but rather, atomic elements of sound which show up in the audio file.

It may be difficult to ascertain exactly *why* the NMF produces this kind of a separation of information. We will explore this concept in more general terms in Section 4.1.3.

In the next section we will take a step back to explore the methods by which the NMF is calculated.

3.3 Calculating the NMF

Recall that the Non-Negative Matrix Factorization seeks matrices V, W, and H, each of which has only non-negative components, such that

$$V = WH.$$

Assuming that the matrix V is of dimension $n \times m$, then the dimension of the matrix W is $n \times r$ and the dimension of H is $r \times m$. The inner dimension, r can be seen as a parameter to be adjusted. Note, that for smaller values of r, it can be seen that an exact factorization $V = WH$ does not exist. Indeed, letting $r = 1$ is the equivalent of factoring a matrix into a column vector and row vector, which is, in general, not possible. In an effort to keep r unrestrained by the possibility of finding an *exact* factorization, it is quite simple to instead look for matrices V, W, and H such that

$$V \approx WH.$$

where the norm of the difference $||V - WH||$ can be restricted as needed. Note that the norm $|| \cdot ||$ is most often taken to be the Frobenius norm, such that

$$||A|| = \sqrt{\sum_{i=1}^{n} \sum_{j=1}^{m} a_{i,j}^2}$$

for a matrix A with elements $a_{i,j}$.

Now that the precise goal has been defined, the algorithm for calculating W and H, given V, can be presented.

The algorithm itself is both simple and provably effective. It begins with the selection of two key parameters: an inner dimension r and a threshold γ, which determines an acceptable norm of the difference $||V - WH||$. The algorithm is iterative, carrying out the same set of instructions repeatedly, until $||V - WH|| \leq \gamma$.

The algorithm begins with the randomization of the components in W and H, ensuring each component is non-negative. Then, the following update is applied iteratively:

$$H \leftarrow H \odot \frac{W^T V}{W^T W H + \epsilon}$$

$$W \leftarrow W \odot \frac{V H^T}{W H H^T + \epsilon}$$

where $\odot$ is element-wise multiplication and $\div$ is element-wise division.

The ϵ term in both denominators refers to a matrix of uniform small values, and is present in the algorithm primarily to avoid division-by-zero errors, or algorithmic divergence due to division by very small numbers.

This algorithm has two important properties which we will discuss in turn: first, elements of W and H are guaranteed to remain non-negative throughout each iteration of the algorithm, and secondly, the norm of the difference $||V - WH||$ is guaranteed to be monotonically decreasing over iterations.

The first property is quite straightforward. Assume V has non-negative entries, and initialize W and H with non-negative entries. In other words, let $V \in \mathbb{R}_{\geq 0}^{n \times m}$, $W \in \mathbb{R}_{\geq 0}^{n \times r}$, and $H \in \mathbb{R}_{\geq 0}^{r \times m}$. Observing the element-wise computation for an element of W:

$$w_{ia} \;\leftarrow\; w_{ia} \, \frac{(VH^T)_{ia}}{(WHH^T)_{ia} + \epsilon},$$

where $i \in \{1, \ldots, n\}$, $a \in \{1, \ldots, r\}$, and $\epsilon > 0$ (interpreted either as a small positive scalar broadcast over the matrix, or as a matrix of small positive entries). Since $V \geq 0$, $H \geq 0$, and $W \geq 0$, we have $(VH^T)_{ia} \geq 0$ and $(WHH^T)_{ia} \geq 0$, hence the ratio is non-negative. Therefore each update multiplies $w_{ia} \geq 0$ by a non-negative quantity, implying the updated w_{ia} remains non-negative.

An analogous computation is carried out for any component of H, and likewise remains non-negative.

The second property, the monotonic decrease of the Frobenius norm $||V - WH||$, requires an examination of the objective function in question.

Specifically, we can let the objective function for this problem take the form

$$f(W, H) = ||V - WH||^2.$$

This objective function is not jointly convex in W and H, but is convex in W when H is held constant (and vice versa) and the non-negativity constraint is enforced. The updates described by Lee and Seung (2000) exploit this fact. They do so by constructing an auxiliary function $G(H, H')$, obtained by replacing the Hessian $\nabla^2 f$ in the second-order Taylor expansion of f around H' with a diagonal matrix $K(H')$ whose entries are chosen large enough to dominate $W^T W$, ensuring the approximation lies everywhere above f. This has the notable property:

$$G(H, H') \geq f(H) \quad \text{and} \quad G(H, H) = f(H)$$

where $f(H) := ||V - WH||^2$ (with W fixed).

Lee and Seung then show that f is non-increasing under the update

$$H^{t+1} = \arg\min_H G(H, H^t)$$

by observing that

$$f(H^{t+1}) \leq G(H^{t+1}, H^t) \leq G(H^t, H^t) = f(H^t)$$

and thereby showing that if the auxiliary function is minimized, the true objective function must monotonically decrease. This shows that the H update monotonically decreases the objective function, and the same logic applies to the W update.

It is important to note that this algorithm does not guarantee convergence to a global minimum. Rather, it only assures that the objective function will not increase over each iteration and tend towards local minima. One's choice of parameters, r and γ, as well as the randomized starting point of the matrices W and H, will play an important role in determining whether the algorithm converges to a suitable minimum.

We have seen how to construct a general Non-negative Matrix Factorization $V \approx WH$. To make the factorization more deterministic (though not completely so), we may wish to sort the rows and columns of W and H respectively. Observe the W matrix in Figure 4.2, which we have previously termed the "Note Matrix". Each column in this matrix corresponds to some note (or notes) which are used as atomic elements, activated by the associated rows in the H matrix. Thus, in order to see the latent structure of W and H more clearly, it helps to sort the columns of the W matrix by frequency. The rows of H can be sorted accordingly so as to preserve their approximation of the matrix V. In the shown figures, a lexicographical sort is implemented from the lowest frequency to the highest frequency. In this sorting scheme, lower frequency columns are placed earlier in the matrix. If two columns in the W matrix have the same lowest active frequency, then higher frequencies are compared to check which column represents a lower frequency in general. As shown in Figure 4.2, this results in an increasing array of individual frequencies, though we note that some "noise" is allowed, where off-diagonal frequencies occasionally exist as fragments of the imperfect factorization of W and H.

Ultimately, this algorithm has produced a particular solution $V \approx WH$ which is sorted predictably and contains interesting latent information that may prove useful for song identification purposes.

3.4 Data Compression

In this section we will analyze the data compression capabilities of the Non-Negative Matrix factorization. Firstly, it is important to note that the NMF is most useful for interpretability rather than its ability to compress data.

A matrix V with dimension $n \times m$ of course has precisely mn components. The factored matrices W and H, on the other hand, are of dimension $n \times r$ and $r \times m$ respectively, so the number of components in the factor matrices is $(nr) + (rm) = r(n + m)$. This gives an explicit expression for the compression ratio c:

$$c = \frac{nm}{r(n+m)}$$

Indeed, for large n and m but small r, c can be quite large, indicating significant compressive ability. The absolute maximum amount of compression possible with the NMF is when $r = 1$, and though r is technically unbounded, it is most often chosen to be less than $\min n, m$.

Of course, the lower r is selected to be, the more lossy the resultant compression. Recall that the NMF seeks an *approximate* factorization $V \approx WH$, so one can expect some reconstruction error determined by the Frobenius norm mentioned in prior sections.

For a more concrete example, we will return to the spectrogram pictured in Figure 2.1, and it's factors in Figures 3.2 and 3.3. In this case, $m = 16513$, $n = 1025$, and $r = 88$. This gives a compression ratio of roughly 11, while maintaining a reconstruction error of 4%. In other words, the matrices W and H only require about 9% of the computer storage space that V requires, while the error in reconstructing V from the matrix multiplication WH remained quite low! At this level of reconstruction error, the differences between the signal represented by V, and its reconstruction, are largely indetectable to the human ear.

While the NMF may be outperformed in terms of compression ratio, reconstructive quality, it remains an important method due to the interpretability it carries alongside its ability to compress data. Of particular note are lossless compression methods like gzip, which are capable of data compression with 0 or near-0 reconstruction error.

3.5 Other Uses of the NMF

Of course, the Non-Negative Matrix Factorization need not be applied only to digital audio. A brief overview of other uses of the NMF follows, as a means of further illustrating the latent structure derived from the NMF, and the usefulness of the technique.

Xu, Liu, and Gong (2003) describes a novel usage of the NMF applied to document clustering. In this paper, the matrix V is conceived as a term-document matrix wherein a dictionary of terms (many thousands of words or short phrases) are counted by their appearance in each document in a given database. Thus V is an $n \times m$ matrix where n is the number of terms in the dictionary, and m is the number of documents in the database. Applying the NMF to this V matrix, Xu, Liu, and Gong (2003) observe the W matrix acts as a "topic basis" matrix describing the building blocks of semantic topics appearing the the corpus of documents, while the H matrix acts as an activation matrix, describing which topics are present in each document. This kind of document clustering is useful when organizing a text database, allowing more efficient browsing and navigation of the corpus, thereby improving the speed and efficacy of information retrieval.

Feng et al. (2022) applies the NMF to the problem of **hyperspectral unmixing**. A hyperspectral image contains hundreds of wavelength measurements per pixel - as opposed to the three or four (R, G, B, and potentially an alpha channel) that one might expect in a traditional image file. There may be as many as 300 spectral bands recorded for each pixel. Often such hyperspectral images are satellite images of Earth taken from such a height so that each pixel may contain an array of physical materials such as soil, vegetation, water, building materials, and more. The measured spectrum of a single pixel is a combination

of the spectra of each material. Crucially, this combination is additive and non-negative. The problem of hyperspectral unmixing thus aims to determine what pure materials are present in the image, where they are, and how much of each is present. Feng et al. (2022) models the spectral image by stacking pixel spectra as columns of a matrix V which has dimension $s \times n$ where s is the number of recorded spectral bands and n is the number of pixels in the image. Then, calculating the NMF $V \approx WH$ one can interpret the W matrix as the "endmember spectra", defining the spectral definitions of pure materials, whereas the H matrix is interpreted as the "abundance vectors", defining how much of each material is present in the corresponding pixel. This paper also considers multiple variants and augmentations to the traditional NMF.

Chapter 4

Source Separation

4.1 Introduction

Most songs are a combination of multiple instruments playing together: the drums keep the beat, the bass lays a melodic foundation, and a guitar, piano, and vocal track round out the piece. The problem of source separation (also called "stem separation") seeks to take a single digital audio recording of a song, where any combination of these instruments may be present, and separate it into constituent source tracks: one for the drums, one for the bass, and so on. The result is a collection of individual files capturing only a single instrument.

There are plenty of uses for such a technology: mixing and mastering a song for digital release often requires access to individual tracks so that instrument recording levels and EQ stay balanced. Remixing songs may require a similar separation of sources so a producer can properly sample some aspect of the song in question. Furthermore, creating a karaoke version of song, with the audio track removed, would be incredibly difficult without source separation.

Luckily, source separation is a well-studied problem. Though the Non-Negative Matrix Factorization discussed in earlier sections can actually be used for source separation,

it is no longer considered the state-of-the-art method. Instead, deep learning-based technologies making use of neural networks and large training datasets have come to the forefront of the field. We will examine a particular tool for source separation called Demucs, examine how it works, and how it could be used to extend the existing scheme of song identification to handle song covers.

4.2 Demucs

Demucs (Deep Extractor for MUltiple Components of Sound) is an open-source music source separation tool introduced in Défossez et al. (2019). The current conception of the technology, a "Hybrid-Transformer" version of Demucs, built upon its predecessor by introducing long-range contextual information using transformer attention mechanisms, and was introduced in Rouard, Massa, and Défossez (2023). For a more thorough introduction to the important role attention plays in transformer architecture, see the seminal paper Vaswani et al. (2017). An emphasis on attention in transformers has shaped the modern conception of many deep learning-based software, and Demucs is no different.

At its core, Demucs is a deep neural network trained to take a single audio waveform as input and produce multiple resultant waveforms as output. The current Hybrid-Transformer model (called Demucs v4) has the following key elements:

1. Encoder-Decoder: Demucs builds on the popular U-Net architecture first introduced by Ronneberger, Fischer, and Brox (2015). U-Net is a popular Convolutional Neural Network which downsamples waveform input to create a compact representation of relevant feature data, learns a compressed representation, then upsamples to produce reconstructed audio sources at full resolution.

2. Hybrid Spectrogram/Waveform Model: Recent improvements combine both waveform-domain and spectrogram-domain processing, meaning both the original digital audio

recording and its spectrogram (derived, for instance, from the STFT) are used when learning representations of features. This gives Demucs the ability to learn from both representations of the input data.

3. Attention Layers: Self-attention and cross-attention layers help the neural network to consider relationships between parts of the song which are distant in time. Critically, a sense of attention in a neural network imbues it with an ability to weigh the relevance of different parts of the song to the task of source separation. This is vital because the sources present in a song generally do not change over time.

4. Training on a large dataset: Demucs v4 was trained on popular supervised datasets containing songs and their respective stems (drums, bass, vocals, other). A large corpus of training data helps Demucs remain versatile, so that it can separate sources from very different kinds of songs.

These features contribute to Demucs improved capability over other source separation techniques like spectral masking (naive masking of specific frequencies in a spectrogram) and traditional approaches like the NMF mentioned previously.

Demucs performance boasts a consistently high Signal-to-Distortion Ratio, a common benchmark which shows that Demucs can separate source instruments from the original signal with clean separation and few artifacts. However, separation quality can vary with music complexity, especially with music that includes instruments dissimilar to those found in training data. Moreover, Demucs outputs only four tracks: *vocals*, *drums*, *bass*, and a catch-all category, *other*.

Demucs usage is quite straightforward. It generally runs as a command-line tool or within a Python library. It can be easily installed from the source or by using the Python package installer, pip, by running

```
pip install demucs
```

and then used in the command line by running

```
demucs PATH_TO_AUDIOFILE
```

which produces four separate audio tracks for each of the sources. Additional options are available for writing to different file formats, separating audio into just vocals and backing track, and more.

In this chapter we have given a short introduction to Demucs source separation. In the next section, we'll observe a usage of Demucs which may help to expand the previously-mentioned *Shazam* fingerprinting algorithm to handle cover and humming detection.

Chapter 5

Cover Detection

5.1 Introduction

Here we present a slightly different problem from the one considered thus far. Imagine you are in a bar which is hosting an open mic night. A local band is covering a song, but they never mentioned the song's title. Perhaps you enjoy their cover, and wish to know the title of the original. Unfortunately, fingerprinting-based song identification scheme presented earlier will inevitably fail in this situation: we can easily extract a set of audio fingerprints from a short recording of the cover, but the corresponding fingerprints will not exist in the database. This highlights a significant limitation to the *Shazam* algorithm presented in earlier sections: it only works reliably for songs that have been previously recorded, fingerprinted, and stored in a database.

In this section, we will discuss extensions and variations of the *Shazam* algorithm which may allow cover detection. We will not start from scratch, but combine the tools already presented - namely, audio fingerprinting and source separation.

5.2 Shortcomings of the Existing Implementation

To identify which song a cover is recreating, one must consider the aspects of a song which are the same between the original and the cover. Lyrics are an obvious answer, since most covers reuse the original lyrics exactly as they are with little change, but are difficult to identify digitally without the help of computationally-expensive speech-to-text machine learning applications, which are most often trained to identify spoken speech rather than sung speech. Instead, we focus on a closely-related aspect: the main melody. In many songs, the main melody is determined by the vocal track, and serves as the main focal point of the song. To illustrate this point, think of your favorite song. If someone hummed that song, or played it on a piano, you would likely recognize the tune despite the lack of lyrics. In this way, the main melody of a song acts as an acoustic foundation, while the other instruments (bass, drums, etc.) serve more predictable roles like keeping a beat (drums) or reinforcing a strong melodic structure (bass).

Discussions of song composition aside, it is clear that the main melody of a cover is an important part of identifying the original song from which it is derived. This idea was explored in detail in Doras and Peeters (2019), which argues that the dominant melody of a song is the single most stable musical element of a song across versions. By isolating and embedding only the dominant melody information, their proposed system achieves competitive cover detection on standard benchmarks.

With the success of Doras and Peeters (2019) in mind, we turn our attention back to fingerprinting. The model of fingerprint extension that we wish to propose builds on the idea of dominant melody isolation, by applying acoustic fingerprinting only to the melody of the cover and the melody of the original song. Provided the cover song is not significantly different from the original song in tempo (time-domain) and key (frequency-domain), we might expect that the *Shazam* algorithm will work just as it did when analyzing the full audio signal.

The problem of clearly identifying and separating the main melody has a number of possible solutions - such as Doras and Peeters (2019) which uses a melody extraction algorithm based on pitch contour characteristics - but the most straightforward is source separation. Thus, a cover-identification version of the *Shazam* algorithm might follow these steps:

1. Build a database of fingerprints, based only on the source-separated vocal track of each song.
2. Before fingerprinting target audio clips, source-separate the vocal track and discard the remaining instrumentation.
3. Calculate the fingerprint of the vocal track of the target song.
4. Compare against the database as normal.

The result is similar to before - fingerprint matching checking hash frequencies and time-offsets - but based on the vocal track, carrying with it the important dominant melody information.

Admittedly, this is a naive approach which ignores a significant fact: singers' voices could have very different timbral detail (imagine the spectral differences between a soprano voice and an alto voice, or the difference between a raspy male voice and a clear female voice). While these may seem unimportant considerations, they will have significant impacts on the constellation map, and therefore on the fingerprint, ruining database match accuracy beyond usability. Indeed, two singers may sing a song at precisely the same tempo, and within the same key, and still produce quite different spectrogram signatures. This poses a problem for cover detection, but not an insurmountable one. In the following subsection, we will see how some straightforward alterations on the existing *Shazam* algorithm can enable cover detection.

5.3 Algorithm Alterations

As discussed, the *Shazam* algorithm is a good starting point for our cover detection algorithm. The following subsections detail each of the alterations one needs to make to the *Shazam* algorithm so that it could theoretically identify cover songs in addition to original recordings.

5.3.1 Demucs source separation

The first, and most significant, alteration to make is to use the Demucs source separator to discard instrumental information, saving only the vocal track. As previously discussed, this allows the algorithm to focus in on the dominant melody without taking into consideration the many background factors that are likely to vary significantly between a cover and the original song. The source separating process is to be carried out both on the database and the recorded clip.

5.3.2 Chromagram

The next alteration to make is to substitute the spectrogram with something called a **chromagram**, which is similar in concept but captures more coarse detail. Rather than using a wide array of frequency bins, the audio signal is sorted into a traditional music scale of 12 distinct frequency bins labelled A, A#, B, C, C#, and so on. Compared to the spectrogram, the chromagram has less finely-grained details. Specifically, the chromagram has no octave information, no timbral details, and no overtones. This allows us to record only the musically-relevant details of the melody (notes and their lengths), while simultaneously ignoring the information which is specific to the individual singer's voice. Thus, qualitative differences in the vocal tracks - like raspiness or octave - will not matter nearly as much as the dominant melody itself.

Importantly, the chromagram is not an altogether different piece of data than the spectrogram. In fact, the chromagram may be computed directly from the spectrogram

by using the following:

$$P = \left\lfloor 12 \log_2 \left(\frac{f}{f_{ref}} \right) \right\rceil \qquad \pmod{12}$$

where P is the pitch class (note), f is the frequency (in Hz) of a given spectrogram bin, and f_{ref} is a reference frequency (often chosen to be 440 Hz). Note that $12 \log_2 \frac{f}{f_{ref}}$ is rounded to the nearest integer to ensure the modularity is well-defined.

This is carried out for each pitch class P, ultimately mapping and spectrogram frequency bins to chromagram pitch classes, and when multiple frequency bins map to a single pitch class, their intensities are summed.

In practice, the chromagram can be computed directly, without using the spectrogram as an intermediary step.

5.3.3 Time Quantization

Another consideration is small micro-differences in timing between the cover and the original. Even accounting for differences in actual tempo, a singer may be temporally early or late by only a few milliseconds - virtually undetectable to the ear, but clearly captured by digital signal processing. As such, the next alteration is to moderately quantize Δt so as to account for these micro-differences. In practice, this means that we pick some quantization factor q so that two measurements of Δt, Δt_1 and Δt_2 are considered equal if $|\Delta t_2 - \Delta t_1| \leq q$.

This quantization helps to make the cover-detection algorithm more robust to very small differences in timing which are to be expected in any cover song, so that covers which take some liberty with timing can still be properly identified.

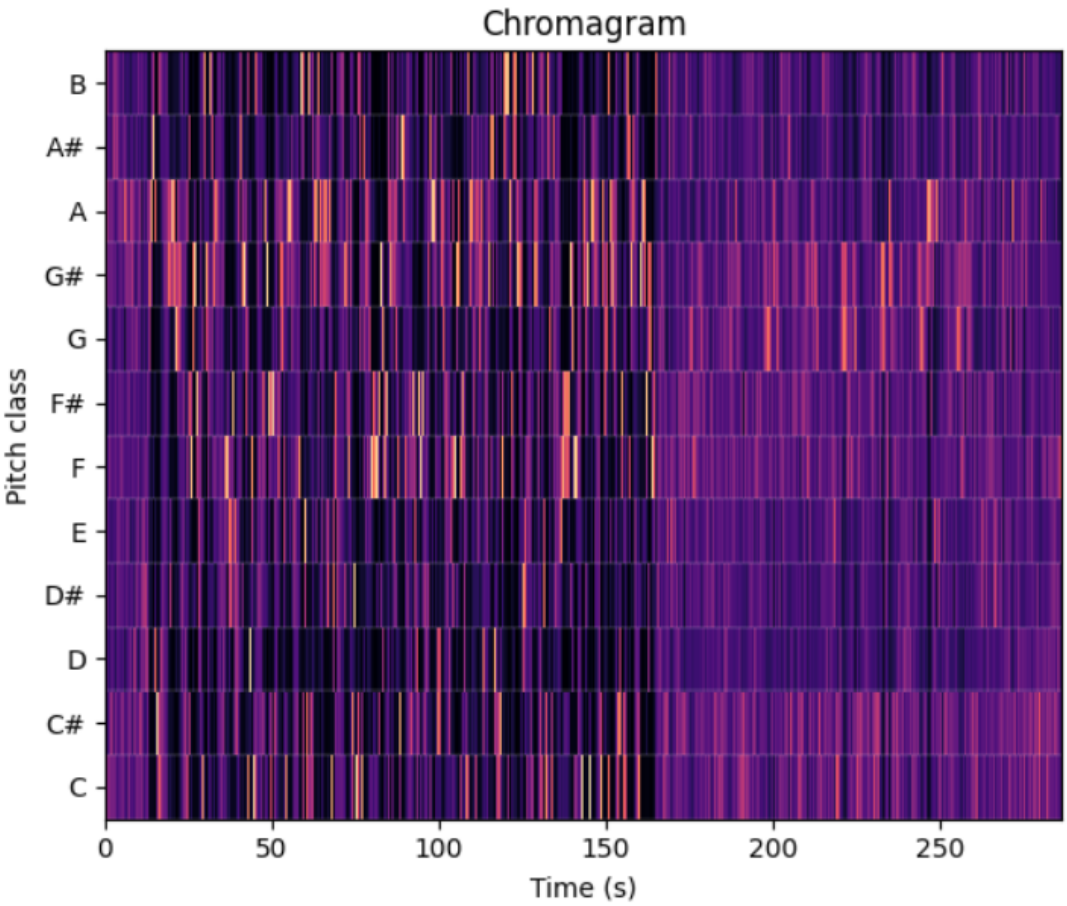

Figure 5.1: The chromagram of a vocal track. Darker values correspond to lower intensity, while lighter values are higher.

5.3.4 Hash Expansion

The change from spectrogram to chromagram has another, unintended effect: with the reduction in frequency bins (from over 1,000 to just 12), the number of possible hashes drops dramatically. Quantizing the time-scale has a similar affect by reducing the number of values for Δt which differentiate one hash from another. This is a problem insofar as the algorithm must retain a certain degree of discriminative power: fewer possible hashes means more hash collisions and more difficulty "seeing through the noise". As such, we must make a change to the structure of hashes in order to regain the discriminative power that was lost.

To understand the change in discriminative power, let us observe it more concretely. Previously, we had hashes of the form $(f_i, f_j, \Delta t)$, and recall that, for a particular f_i, we restrict the values of f_j and Δt so that f_j lies in a shifted window, close to (in both frequency and time) the location of f_i. Thus we can define some range of values, in terms of frequency and time bins, that determines the total number of possible hashes.

If M is the number of possible values for f_i, β is the number of permissible frequency bins that f_j can inhabit, and α is the number of possible values for Δt, then the hash space holds $M\alpha\beta$ possibilities. Practically, we often set these values to something on the order of: $M = 8000$, $\beta = 1000$ and $\alpha = 200$, giving 1.6 billion possibilities. Similar values to these are chosen for the example shown in Figure 2.3.

Transitioning to the new cover-detection scheme requires that these values all become much lower, such that $M = 12$, $\beta \leq 12$ (since there are only 12 pitch classes and we may choose to restrict the acceptable frequency-range even further if needed) and, quantizing by a factor of 5, $\alpha = 40$. This brings the total hash space to just $M\alpha\beta \leq 5760$, which is sure to cause significant hash collision issues in any practical scenario. Thus, we must make some change to increase the hash space and restore discriminative power to the algorithm - otherwise, a large database will not be able to determine matches, since each query against the database will return many matches.

The remedy to the hash collision problem is this: we re-define a hash to be a set of 3 frequencies with 2 offsets. Thus we have 5-component hashes of the form $(f_i, f_j, f_k, \Delta t_j, \Delta t_k)$, where f_i is still the anchor point, and f_j and f_k both reside within a shifted window. This is a natural extension of the previous conception of a hash - in essence, we are combining two hashes together which share an anchor point f_i, so that each hash captures more specific local information about the constellation map. This gives a much bigger hash space of up to about $(12^3)(40^2) \approx 2.76$ million. It may also be practical to increase the value of α by up to two times, such that the hash space extends even further to

$(12^3)(80^2) \approx 1.1$ billion, which is closer to the discriminative power achieved by the original algorithm.

This change is made primarily to remedy the loss in discriminative power caused by the previous two alterations, but also has the secondary result of capturing finer local details. Each expanded hash captures more specific information about the local structure of the chromagram than the 2-frequency hash as implemented in Wang (2003).

5.3.5 Other alterations

A few other alterations can be made to ensure the algorithm works reasonably well for cover detection. While not completely necessary, these alterations will result in an algorithm which more closely aligns with the underlying musical theory, and permit reasonable differences in tempo between an original song and its cover.

Firstly, we introduce the idea of chromagram wraparound distance. Since all frequency bins present in the spectrogram are collapsed into chromagram pitch classes, it is important to account for the repeating structure of the traditional 12-tone musical scale. In Figure 5.1, pitch classes for C and B are on opposite ends of the pitch class axis. However, in musical theory, C and B are considered adjacent notes, and chromagram wraparound distance allows us to account for this fact. Anchor points close to the lower or upper bounds of the axis (close to C or B pitch classes, respectively) should include in their target window target points on the other end of the pitch class axis. Intuitively, this means that the target window "loops around" to the other side when it reaches or surpasses the bounds of the pitch class axis. This way, pitch classes C and D are considered close, and the algorithm avoids an arbitrary separation between these two notes. One can imagine that failing to account for this might not have catastrophic consequences for the performance of the algorithm, but making this alteration ensures that the practical application of the algorithm aligns well with the underlying music theory.

Lastly, we wish to account for overall tempo differences between the cover song and original recording. Consider a song originally recorded at 120 beats per minute (bpm). Studio recordings are often standardized to bpm measurements in multiples of 10, however, live performances of these songs by cover bands are not likely to adhere closely to these bpm measurements. Even if the cover bands intends to make a fully faithful recreation of the original song, achieving a perfect bpm-match is difficult without the help of a metronome or other tempo-keeping device. Such differences in tempo may be quite unnoticeable to the human ear, but they will certainly show up in the acoustic fingerprint of the cover song recording. In short, one can expect a cover song to have a similar - but not exactly the same - tempo as compared to the original song. To account for this, we return to the database absolute time vs. recording absolute time comparison from Subsection 2.5.2. Recall that, in the original *Shazam* algorithm, we require hash matches (matches between our short recording and the song in the database) to have a consistent time offset. In other words, the *absolute time* relation between candidate hashes should be exactly consistent. If two candidate hashes have a time-difference of t in the recording, they should have that same time-difference t in the database. In Figure 2.4, this was visualized as matched hashes lying on a line with slope of exactly 1. In our cover detection algorithm, we allow this relation to be somewhat more lenient: while we still require a linear relationship between candidate hashes' recording times and database times, the slope may exist within a range centered on 1. For instance, allowing a slope between 0.9 and 1.1 permits a song originally recorded at 120 bpm to be identifiable as a cover anywhere between 108 and 132 bpm.

5.4 The Algorithm Implemented

5.4.1 A toy example

In this section, we will show that the cover detection algorithm described in previous sections works consistently on a toy-scale database of 5 songs. As described, we choose a corpus of songs to be a part of the database and carry out the acoustic fingerprinting process on each of them, storing in a database a list of hashes, along with their respective song IDs and timestamps.

For our toy database, we chose the following 5 songs:

- "Breakadawn" - *De La Soul*
- "Fire in the Hole" - *Steely Dan*
- "Life on Mars" - *David Bowie*
- "The Girl From Ipanema" - *Stan Getz and Astrud Gilberto*
- "What You Won't Do for Love" - *Bobby Caldwell*

These 5 songs were chosen due to their vocal-driven melodies, meaning that they are well-suited for the cover detection algorithm which relies on the fact that the vocal track carries the dominant melody. Of course, with more instrumentally-focused songs, the cover detection algorithm would perform poorly.

The cover chosen for the toy example was an acoustic (guitar and vocals only) cover of "The Girl From Ipanema", performed by Josh Turner and Carson McKee. A section of the cover song (for instance, the middle 30 seconds) is chosen as our "field recording" which will be fingerprinted to be used for comparison against the database.

Our toy implementation follows the exact process described above. To give a quick summary:

1. The song's vocals are separated and the instrumentation is discarded.

2. A chromagram is computed from the vocal track.

3. Local maxima within the chromagram are identified, forming the constellation map.

4. 3-frequency hashes are computed from the constellation map.

This process is carried out for each song in the database first, and then for the cover song.

Some parameters of the algorithm are easily tunable to achieve slightly different results in the outcome of cover identification. For transparency and reproducibility, the exact parameters, along with descriptions and justification of the chosen value, can be found in Table 5.1.

The chosen parameters essentially determine the efficacy of the algorithm. Different sets of parameters may be chosen for different amounts of leniency when detecting covers. In essence, these parameters give the programmer a quantifiable way to answer the question, "how different will I allow a potential cover to be from the original song?".

Tuning these parameters is an iterative process rather than an analytic one. The values displayed in the table were shown to be effective only through a trial-and-error process, producing the following results.

Given the 5 database songs and the cover song, the algorithm works as described and produces a "best guess" in the form shown in Table 5.2. This shows the following for each song in the database:

- Song: The name of the database song.

- Aligned: The number of hash matches whose time-offsets lie on a straight line within the acceptable slope range (discussed in subsection 5.3.5).

- Wt. Scr: The weighted score calculated from the number of aligned hashes, weighted using an Inverse Document Frequency scheme (common hashes in the database have lower weights, rare hashes have higher weights).

- Norm: The Normalized score. This is calculated by multiplying the Wt. Scr by a penalty based on time-alignment (tempo differences between cover and original produce a small penalty) and then dividing by the number $\sqrt{N_{db}N_{query}}$ where N is the number of hashes in a match. This normalization encodes the belief that cover songs tend to be similar in tempo to their original, and normalizes for differences in fingerprint count.

- DB hashes: The total number of hashes for a given song. Notice in Table 5.2 the difference in DB hashes, necessiting the normalization mentioned in the previous bullet.

- Slope: Estimated tempo ratio between the query and the database song. A slope of 1 is "ideal".

Ultimately, only the normalized score in Table 5.2 is the deciding factor when identifying an ultimate choice - the algorithm simply chooses the highest score. In this case, cover detection was successful by a decent margin: the correct song ("The Girl From Ipanema") had a normalized score of ≈ 0.005, where the next-highest option has a much smaller score, ≈ 0.001.

5.4.2 Shortcomings

We have presented a proof-of-concept for the cover detection algorithm described in previous sections, but of course this is only a toy example. It shows that the ideas work on a small scale, but any reasonable database would be significantly larger than the modest 5 songs presented here. Indeed, an industrial-scale implementation of this cover detection algorithm would require more fine-tuning of parameters, and would likely require additional considerations.

A notable issue is the tendency of musicians to re-use melodies, further complicating matters of song detection. Two songs may sound quite different but have similar dominant

melodies, posing a problem for a dominant melody-focused algorithm. In a database with many such songs, a ranked output of "song match" likelihoods may prove more useful to users of the technology than a single-song output.

Another aspect of the cover detection algorithm worth mentioning is that it requires users to take longer recordings than the original *Shazam* algorithm. This is partly by design: rather than focusing on very fine frequency details present in the spectrogram, the cover detection algorithm detects melodic changes in the chromagram. For a collection of fingerprints to describe melodic changes effectively, the recording should ideally be long enough to capture multiple notes - whereas in *Shazam's* case, the recording can be comparatively shorter since it is capturing finer, more timbral details.

We consider it likely that some other data representation - perhaps derived from the Non-Negative Matrix Factorization of an audio recording's spectrogram - would be required when implementing this particular algorithm on a large scale. More research, both theoretically and experimentally, is required to understand how the algorithm presented in this section scales. Luckily, this work is being actively carried out by the scientific community, and in the next section we will present a brief survey of recent results.

5.5 Other routes of research

An important part of connecting a cover song to its original is largely ignored by the algorithm presented in this chapter: lyrics. Cover artists often leave lyrics completely unchanged in their versions, even if the musical quality of the cover is quite different from the original. Correya, Hennequin, and Arcos (2018) discuss how using multiple kinds of data, crucially including lyrics, can significantly improve cover detection outcomes. By combining the use of lyrics, metadata, and audio, Correya, Hennequin, and Arcos (2018) saw significant improvements over the state-of-the-art of the time.

Chang et al. (2017) and Yu et al. (2020) both apply machine learning to the problem of cover detection more directly. In Chang et al. (2017) a Convolutional Neural Network (CNN) is constructed to take input in the form of a cross-similarity matrix between two songs. In turn, it outputs the likelihood that they have a cover-original relationship. This reduces the problem of song detection to a pair-wise binary classification problem. In other words, it takes in information from two songs and seeks to determine whether they are related. Yu et al. (2020) also proposes a CNN architecture, but in this paper, the problem is framed as a representation embedding problem. Songs are embedded as vectors in a high-dimensional representation space, and cover-original relationships are then derived from distances between these vectors.

For a more comprehensive overview of the field of cover detection, Serrà, Gómez, and Herrera (2010) is a well-cited and foundational survey of the field. Rather than promising the best techniques, it can provide the reader a foundational understanding of the challenges in cover detection, which are still perfectly relevant today. Many of the challenges presented in Serrà, Gómez, and Herrera (2010) are the very same challenges encountered in this chapter: the need to be robust to tempo changes, timbral differences, etc. These are fundamentally difficult problems to solve, and more work is needed to push the state-of-the-art forward.

Table 5.1: Tunable parameters of the cover-detection algorithm.

Parameter & Value	Description	Justification
`sample_rate`= 22 050 Hz	Sample rate. The rate at which all audio files are loaded via `librosa.load`, a Python audio loading library.	This is the standard sample rate in Music Information Retrieval (MIR), capturing all relevant pitch information at half the computational cost of CD-quality audio.
`neighborhood_size`= $(5, 30)$	Peak window. The size of the local neighborhood (chroma bins $\times$ time frames) used by the maximum filter when detecting peaks in the chromagram.	A width of 5 covers nearly half the 12-bin chroma axis, while a span of 30 frames (≈ 0.7 s) limits the constellation to roughly one peak per pitch class per musical beat.
`threshold`=0.15	Peak amplitude threshold. The minimum chromagram energy a bin must have (after normalization) to qualify as a peak.	After normalization, values range from 0 to 1. A threshold of 0.15 filters noise while retaining harmonic content.
`max_peaks`=500	Constellation points cap. The maximum number of peaks retained per song, selected in order of descending amplitude.	For a typical 3–5 minute song this yields roughly one peak per beat, providing sufficient density for robust matching while keeping computation costs reasonable.
`delta_t_min`=5 frames	Minimum time gap. The minimum separation in frames (≈ 116 ms) between consecutive points in a triplet; closer pairs are discarded.	Very close points likely represent the same musical event or spectral artefact; enforcing a minimum gap ensures paired points capture distinct temporal events.
`delta_t_max`= 200 frames	Maximum time gap. The maximum separation in frames (≈ 4.6 s) between consecutive points in a triplet; more distant pairs are discarded.	Beyond roughly one musical phrase the timing relationship between two points becomes unreliable as a fingerprint. This also bounds the quantised time-difference axis to $200/5 = 40$ bins.
`dt_quantize` = 5 frames	Time-difference quantisation. The integer divisor applied to each time-difference component of a hash, grouping nearby values into the same bucket (≈ 116 ms per bucket).	Quantisation provides tolerance for local timing variations (rubato, slight tempo differences) while preserving enough resolution to keep the hash space discriminative ($12 \times 12 \times 12 \times 40 \times 40 \approx 2.76$ million unique hashes).

Table 5.2: Cover detection results for *The Girl From Ipanema* (cover query).

Song	**Aligned**	**Wt. Scr**	**Norm**	**DB hashes**	**Slope**
The Girl From Ipanema	109	269.5	0.00530	79 847	1.00
Fire In The Hole	25	62.1	0.00135	61 291	0.96
De La Soul – Breakadawn	18	43.8	0.00119	39 047	0.96
Life On Mars	18	44.2	0.00105	54 353	1.02
What You Won't Do for Love	13	29.1	0.00083	35 023	0.96

Bibliography

Chang, Sungkyun et al. (2017). "Audio Cover Song Identification using Convolutional Neural Network". In: *arXiv preprint arXiv:1712.00166*. URL: https://arxiv.org/abs/1712.00166.

Cooley, James W. and John W. Tukey (1965). "An algorithm for the machine calculation of complex Fourier series". In: *Mathematics of Computation* 19.90, pp. 297–301.

Correya, Albin Andrew, Romain Hennequin, and Josep Lluis Arcos (2018). "Large-Scale Cover Song Detection in Digital Music Libraries Using Metadata, Lyrics and Audio Features". In: *arXiv preprint arXiv:1808.10351*. URL: https://arxiv.org/abs/1808.10351.

Défossez, Alexandre et al. (2019). "Music Source Separation in the Waveform Domain". In: *arXiv preprint arXiv:1911.13254*.

Doras, Guillaume and Geoffroy Peeters (2019). "Cover Detection Using Dominant Melody Embeddings". In: *Proceedings of the 20th International Society for Music Information Retrieval Conference (ISMIR)*. Delft, Netherlands, pp. 107–114.

Feng, Xin-Ru et al. (2022). "Hyperspectral Unmixing Based on Nonnegative Matrix Factorization: A Comprehensive Review". In: *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 15, pp. 4414–4431. DOI: 10.1109/JSTARS.2022.3174126.

Grondin, François, Paul Jolicoeur-Martineau, Pierre-Antoine Noël, et al. (2020). "Neural Audio Fingerprinting". In: *Proceedings of ICASSP*.

Haitsma, Jaap and Ton Kalker (2002). "A Highly Robust Audio Fingerprinting System". In: *Proceedings of the International Conference on Music Information Retrieval (ISMIR)*.

Lee, Daniel D. and H. Sebastian Seung (1999). "Learning the parts of objects by non-negative matrix factorization". In: *Nature* 401.6755, pp. 788–791. DOI: 10.1038/44565.

— (2000). "Algorithms for Non-negative Matrix Factorization". In: *Advances in Neural Information Processing Systems*. Vol. 13, pp. 556–562.

Ronneberger, Olaf, Philipp Fischer, and Thomas Brox (2015). "U-Net: Convolutional Networks for Biomedical Image Segmentation". In: *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*. Vol. 9351. LNCS. (available on arXiv:1505.04597 [cs.CV]). Springer, pp. 234–241.

Rouard, Simon, Francisco Massa, and Alexandre Défossez (2023). "Hybrid Transformers for Music Source Separation". In: *ICASSP 2023*. IEEE.

Serrà, Joan, Emilia Gómez, and Perfecto Herrera (2010). "Audio Cover Song Identification and Similarity: Background, Approaches, Evaluation, and Beyond". In: *Advances in Music Information Retrieval*. Vol. 274. Studies in Computational Intelligence. Springer, pp. 307–332. DOI: 10.1007/978-3-642-11674-2_14.

Solat, Siamak (2023). "Sharding Distributed Databases: A Critical Review". In: *arXiv preprint arXiv:2304.11523*. Available at: https://arxiv.org/abs/2304.11523.

Vaswani, Ashish et al. (2017). "Attention is all you need". In: *Advances in neural information processing systems*, pp. 5998–6008.

Wang, Avery Li-Chun (2003). "An Industrial-Strength Audio Search Algorithm". In: *Proceedings of the International Conference on Music Information Retrieval (ISMIR)*.

Wold, Erik et al. (1996). "Content-Based Classification, Search, and Retrieval of Audio". In: *IEEE Multimedia* 3.3, pp. 27–36.

Xu, Wei, Xin Liu, and Yihong Gong (2003). "Document Clustering Based on Non-negative Matrix Factorization". In: *Proceedings of the 26th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '03)*. Toronto, Canada: ACM, pp. 267–273.

Yu, Zhesong et al. (2020). "Learning a Representation for Cover Song Identification Using Convolutional Neural Network". In: *ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Barcelona, Spain, pp. 541–545. DOI: 10.1109/ICASSP40776.2020.9053839.